

CSE344 – Systems Programming

Homework 6

Magic Academy (Hogwarts) System

TCP Server-Client

Single-Threaded I/O Multiplexing
with `select()`

Student Name: Salih Cengiz
Student ID: 220104004007
Course: CSE344 – Systems Programming
Assignment: Homework 6
Date: May 16, 2026

Contents

1	System Design	2
1.1	Architecture Overview	2
1.2	Single-Threaded Select Loop	2
1.3	Idle Timeout Mechanism	3
1.4	Capacity Enforcement	3
2	Data Structures	4
2.1	Client State (<code>client_t</code>)	4
2.2	Ingredient Storage	4
2.3	Spellbook Management	4
3	Partial Read/Write Handling	5
3.1	Partial Read (Line Buffering)	5
3.2	Partial Write	5
4	Signal Handling and Shutdown	5
4.1	Server Shutdown (SIGINT)	5
4.2	Client Exit (Ctrl+C)	6
4.3	SIGPIPE Handling	6
5	Test Scenarios	6
5.1	Scenario 1 — Unauthorized Commands, Duplicate Username, Unknown Commands	7
5.2	Scenario 2 — BREW, CONSUME, SPELLBOOK, Error Cases	9
5.3	Scenario 3 — Capacity Enforcement and Slot Recovery	11
5.4	Scenario 4 — Idle Timeout and Server Shutdown	13
6	Challenges Encountered	16
6.1	Challenge 1: SIGPIPE Crash on Client Disconnect	16
6.2	Challenge 2: Timing-Sensitive Idle Timeout Logic	16
6.3	Challenge 3: Double APPARATE on EOF	17
7	Compilation	17

1 System Design

1.1 Architecture Overview

The Magic Academy system consists of three programs:

- **hogwarts** (server): A single-threaded TCP server that manages all client connections using `select()` I/O multiplexing.
- **wizard** (client): Connects to the server as a WIZARD type, can BREW, CONSUME, and check SPELLBOOK.
- **professor** (client): Connects to the server as a PROFESSOR type, can INSPECT ingredients, view SCROLL (all ingredients), and check ROSTER (connected clients).

1.2 Single-Threaded Select Loop

The server manages all TCP clients within a single event loop without any threading. The design is as follows:

1. The server creates a listening TCP socket and binds to the specified port.
2. In each iteration of the main loop:
 - (a) **Timeout Check:** Iterate over all connected clients and disconnect any whose `last_active` timestamp exceeds the configured timeout.
 - (b) **Select Timeout Calculation:** Calculate the minimum remaining idle time across all clients and use this as the `select()` timeout value. This ensures timely timeout detection without `sleep()` or threads.
 - (c) **fd_set Construction:** Add the listening socket and all client sockets to the read `fd_set`.
 - (d) **select() Call:** Block until data is available or timeout expires.
 - (e) **New Connections:** If the listening socket is readable, call `accept()`. If `num_connected >= max_clients`, send `ERR HOGWARTS_FULL` and close immediately. Otherwise, assign a slot and initialize client state.
 - (f) **Client Data:** For each readable client socket, read data into a per-client line buffer. When a complete line (ending with `\n`) is found, parse and process the command.

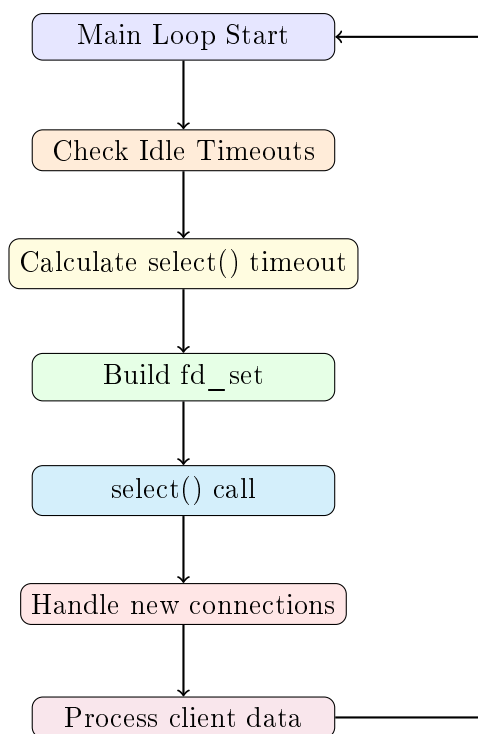


Figure 1: Server Main Loop Flow

1.3 Idle Timeout Mechanism

The idle timeout is enforced without `sleep()` or threads. Before each `select()` call, the server:

1. Iterates all connected clients to find `min_remaining = timeout_sec - (now - client.last_act`
2. Sets `select()`'s `struct timeval` to this minimum value.
3. After `select()` returns (either due to data or timeout), checks all clients again for expiration.

This guarantees that no client can remain idle longer than the configured timeout, with precision to within 1 second.

1.4 Capacity Enforcement

When `max_clients` is reached:

- The server still calls `accept()` to avoid kernel backlog buildup.
- Immediately sends `ERR HOGWARTS_FULL` to the new connection.
- Closes the file descriptor **without** adding it to the client array.
- Logs `REJECTED fd=N ip=X reason=HOGWARTS_FULL`.

When a client disconnects, its slot is freed (`fd = -1`), allowing new connections to reuse it.

2 Data Structures

2.1 Client State (client_t)

```
1 typedef struct {
2     int     fd;                // TCP socket fd (-1 if free)
3     char    username[32];      // registered name
4     char    type[16];          // "WIZARD" or "PROFESSOR"
5     char    line_buf[512 + 1]; // partial read buffer
6     int     buf_len;           // current buffer length
7     time_t  last_active;       // last command timestamp
8     int     enrolled;          // 1 after successful ENROLL
9     spellbook_entry_t spellbook[64]; // wizard spellbook
10    int     spellbook_count;    // number of entries
11 } client_t;
```

The server allocates an array of `max_clients` elements. A slot with `fd == -1` is considered free.

2.2 Ingredient Storage

```
1 typedef struct {
2     char name[17]; // e.g., "MOONSTONE"
3     int  quantity; // current global quantity
4 } ingredient_t;
```

Ingredients are loaded at startup from `ingredients.txt`. The global array is shared across all clients.

2.3 Spellbook Management

Each wizard maintains a personal spellbook tracking how much of each ingredient they have brewed:

```
1 typedef struct {
2     char name[17];
3     int  quantity; // amount brewed by this wizard
4 } spellbook_entry_t;
```

- **BREW**: Adds to both the global ingredient quantity and the wizard's spellbook entry.
- **CONSUME**: Checks the wizard's spellbook first (must have brewed enough), then deducts from both spellbook and global quantity.
- **SPELLBOOK**: Lists all entries with quantity > 0 in the wizard's spellbook.
- On disconnect, the spellbook is discarded (not persisted).

3 Partial Read/Write Handling

3.1 Partial Read (Line Buffering)

TCP is a stream protocol — a single `read()` may return partial lines, multiple lines, or combinations thereof. The server handles this with a per-client `line_buf`:

1. `read()` returns raw bytes into a temporary buffer.
2. Each byte is appended to the client's `line_buf`.
3. When a newline (`\n`) is encountered, the accumulated line is passed to `process_command()`, and the buffer is reset.
4. If the buffer exceeds 512 bytes without a newline, the server sends `ERR TOOLONG` and resets the buffer.

3.2 Partial Write

The `send_to_client()` function loops until all bytes are written:

```
1 static int send_to_client(int fd, const char *data, size_t len) {
2     size_t sent = 0;
3     while (sent < len) {
4         ssize_t n = write(fd, data + sent, len - sent);
5         if (n < 0) {
6             if (errno == EINTR) continue;
7             return -1;
8         }
9         sent += (size_t)n;
10    }
11    return 0;
12 }
```

This handles the case where the kernel buffer is full and `write()` sends fewer bytes than requested.

4 Signal Handling and Shutdown

4.1 Server Shutdown (SIGINT)

1. A `SIGINT` handler sets volatile `sig_atomic_t shutdown_flag = 1` using only async-signal-safe operations.
2. The main `select()` loop checks `shutdown_flag` at the top of each iteration and after `select()` returns with `EINTR`.
3. When detected:
 - (a) Logs `SHUTDOWN signal=SIGINT`.
 - (b) Sends `SERVER SHUTDOWN` to every connected client.
 - (c) Closes all client file descriptors.

- (d) Logs `CLEANUP_DONE` `clients=0`.
- (e) Closes the listening socket, frees memory, and exits with code 0.

4.2 Client Exit (Ctrl+C)

Both wizard and professor clients:

1. Install a `SIGINT` handler that sets `volatile sig_atomic_t quit_flag = 1`.
2. The main `select()` loop detects the flag.
3. Sends `APPARATE` to the server.
4. Reads the `OK APPARATE` response.
5. Prints `DISCONNECTED reason=APPARATE` and exits cleanly.

4.3 SIGPIPE Handling

The server calls `signal(SIGPIPE, SIG_IGN)` at startup. Without this, writing to a socket whose client has already closed would generate `SIGPIPE`, terminating the server. By ignoring it, `write()` simply returns `-1` with `errno = EPIPE`, which the server handles gracefully.

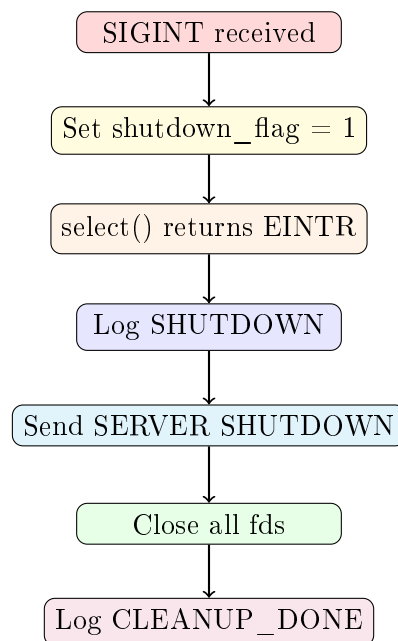


Figure 2: Server SIGINT Shutdown Sequence

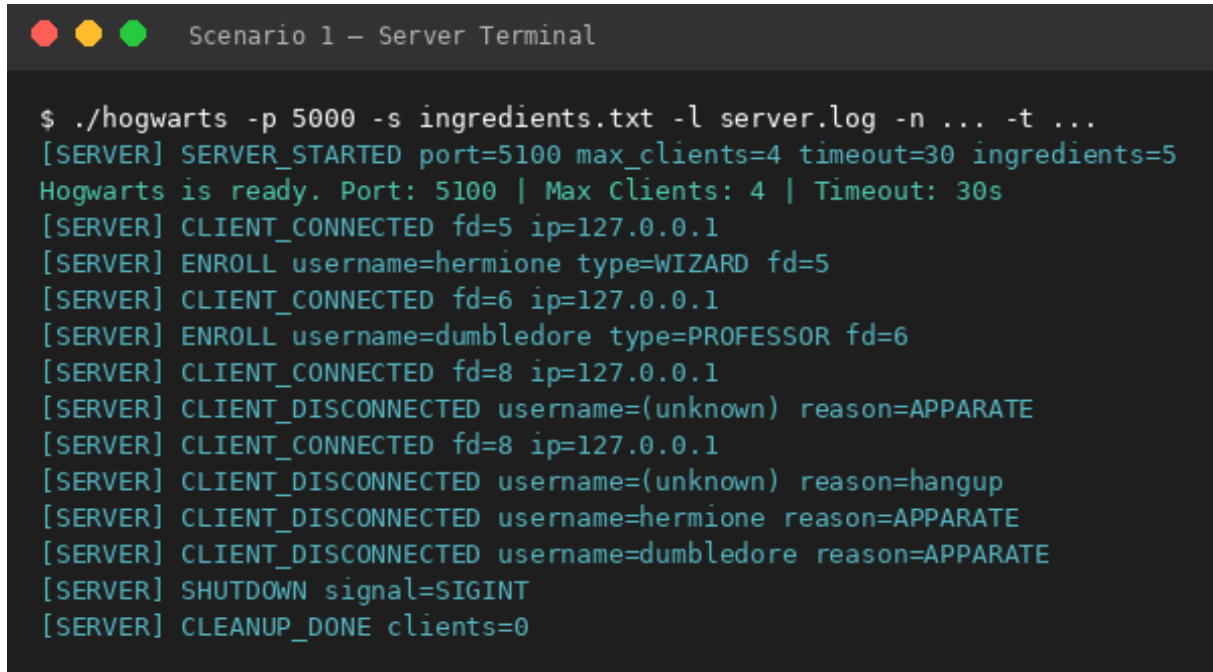
5 Test Scenarios

All four mandatory test scenarios were executed on Ubuntu (WSL2) with FIFO-based multi-terminal simulation. Each terminal's uncut output is captured below.

5.1 Scenario 1 — Unauthorized Commands, Duplicate Username, Unknown Commands

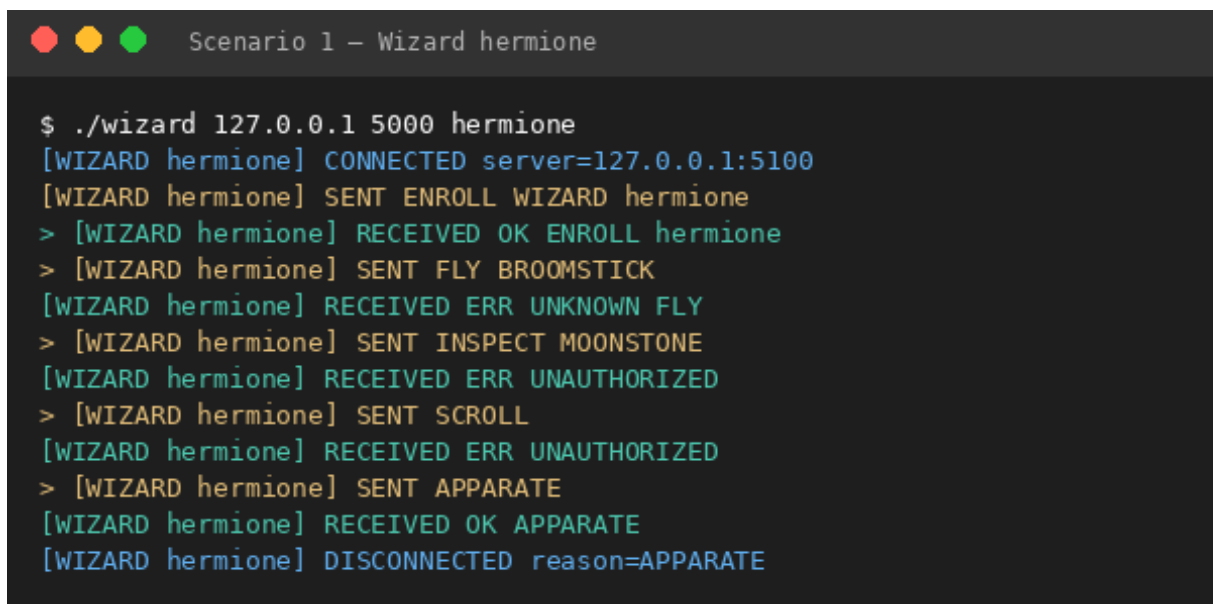
Server: `./hogwarts -p 5100 -s ingredients.txt -l server_s1.log -n 4 -t 30`

Tests: duplicate username rejection, command before ENROLL, unknown command (FLY BROOMSTICK), unauthorized commands (professor BREW/CONSUME, wizard INSPECT/SCROLL), Ctrl+C clean disconnect.



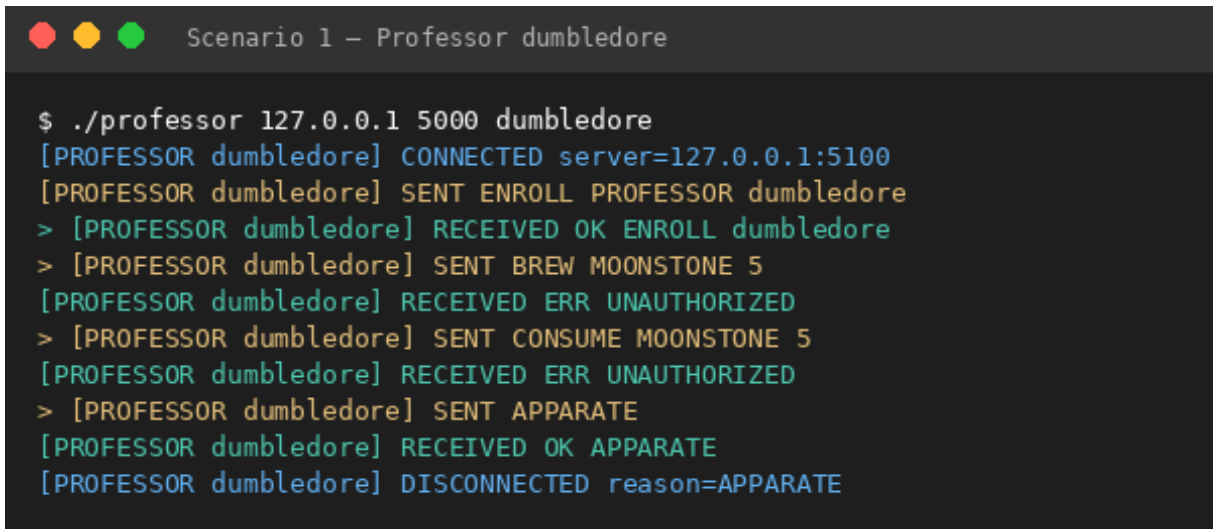
```
$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n ... -t ...
[SERVER] SERVER_STARTED port=5100 max_clients=4 timeout=30 ingredients=5
Hogwarts is ready. Port: 5100 | Max Clients: 4 | Timeout: 30s
[SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[SERVER] ENROLL username=hermione type=WIZARD fd=5
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] ENROLL username=dumbledore type=PROFESSOR fd=6
[SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[SERVER] CLIENT_DISCONNECTED username=(unknown) reason=APPARATE
[SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[SERVER] CLIENT_DISCONNECTED username=(unknown) reason=hangup
[SERVER] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0
```

Figure 3: Scenario 1 — Server Terminal



```
$ ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5100
[WIZARD hermione] SENT ENROLL WIZARD hermione
> [WIZARD hermione] RECEIVED OK ENROLL hermione
> [WIZARD hermione] SENT FLY BROOMSTICK
[WIZARD hermione] RECEIVED ERR UNKNOWN FLY
> [WIZARD hermione] SENT INSPECT MOONSTONE
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
> [WIZARD hermione] SENT SCROLL
[WIZARD hermione] RECEIVED ERR UNAUTHORIZED
> [WIZARD hermione] SENT APPARATE
[WIZARD hermione] RECEIVED OK APPARATE
[WIZARD hermione] DISCONNECTED reason=APPARATE
```

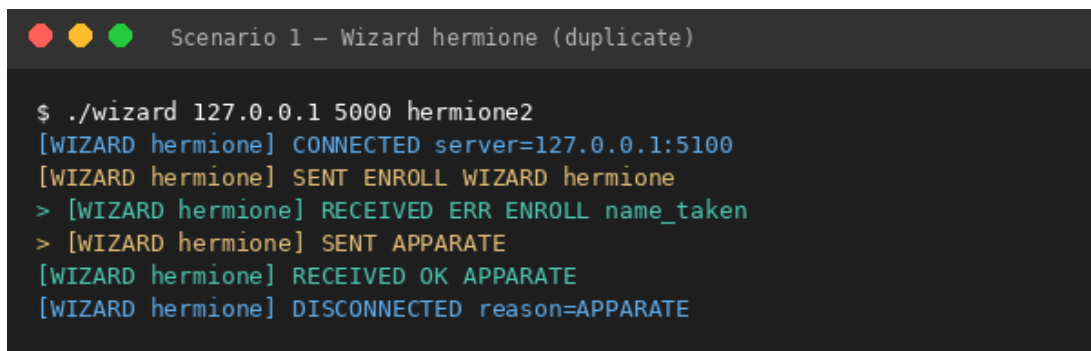
Figure 4: Scenario 1 — Wizard hermione (FLY BROOMSTICK → ERR UNKNOWN, INSPECT/SCROLL → ERR UNAUTHORIZED)



```
Scenario 1 - Professor dumbledore

$ ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5100
[PROFESSOR dumbledore] SENT ENROLL PROFESSOR dumbledore
> [PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
> [PROFESSOR dumbledore] SENT BREW MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
> [PROFESSOR dumbledore] SENT CONSUME MOONSTONE 5
[PROFESSOR dumbledore] RECEIVED ERR UNAUTHORIZED
> [PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE
```

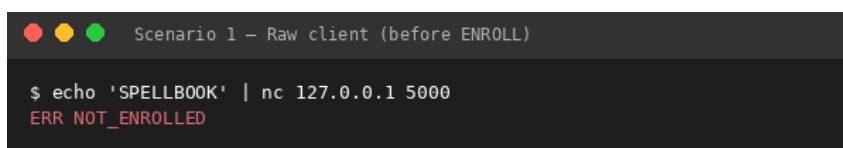
Figure 5: Scenario 1 — Professor dumbledore (BREW/CONSUME → ERR UNAUTHORIZED)



```
Scenario 1 - Wizard hermione (duplicate)

$ ./wizard 127.0.0.1 5000 hermione2
[WIZARD hermione] CONNECTED server=127.0.0.1:5100
[WIZARD hermione] SENT ENROLL WIZARD hermione
> [WIZARD hermione] RECEIVED ERR ENROLL name_taken
> [WIZARD hermione] SENT APPARATE
[WIZARD hermione] RECEIVED OK APPARATE
[WIZARD hermione] DISCONNECTED reason=APPARATE
```

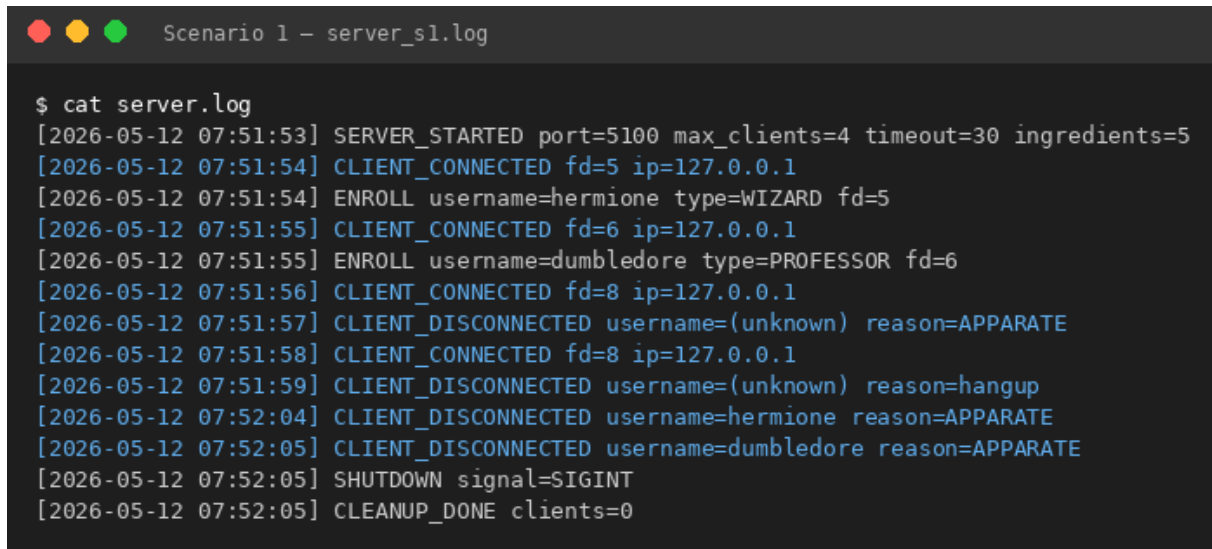
Figure 6: Scenario 1 — Duplicate hermione → ERR ENROLL name_taken



```
Scenario 1 - Raw client (before ENROLL)

$ echo 'SPELLBOOK' | nc 127.0.0.1 5000
ERR NOT_ENROLLED
```

Figure 7: Scenario 1 — Raw client sends SPELLBOOK before ENROLL → ERR NOT_ENROLLED



```

Scenario 1 - server_s1.log

$ cat server.log
[2026-05-12 07:51:53] SERVER_STARTED port=5100 max_clients=4 timeout=30 ingredients=5
[2026-05-12 07:51:54] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-12 07:51:54] ENROLL username=hermione type=WIZARD fd=5
[2026-05-12 07:51:55] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-12 07:51:55] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-12 07:51:56] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-12 07:51:57] CLIENT_DISCONNECTED username=(unknown) reason=APPARATE
[2026-05-12 07:51:58] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-12 07:51:59] CLIENT_DISCONNECTED username=(unknown) reason=hangup
[2026-05-12 07:52:04] CLIENT_DISCONNECTED username=hermione reason=APPARATE
[2026-05-12 07:52:05] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-12 07:52:05] SHUTDOWN signal=SIGINT
[2026-05-12 07:52:05] CLEANUP_DONE clients=0

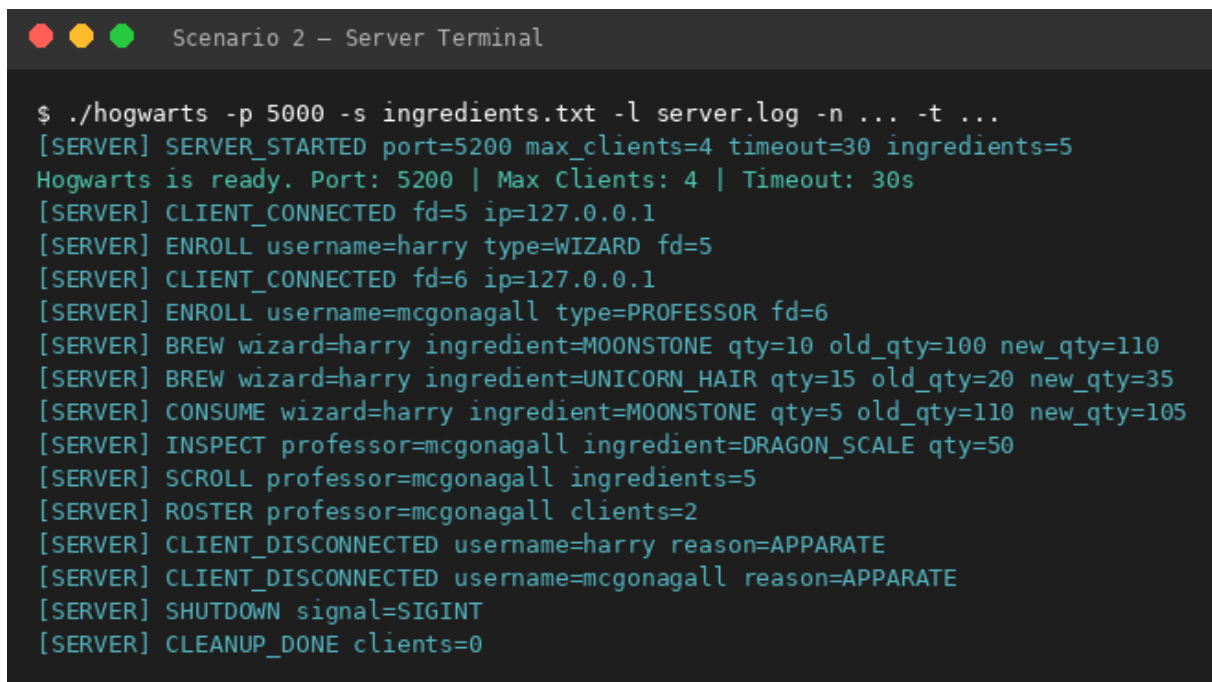
```

Figure 8: Scenario 1 — Server Log (server_s1.log)

5.2 Scenario 2 — BREW, CONSUME, SPELLBOOK, Error Cases

Server: `./hogwarts -p 5200 -s ingredients.txt -l server_s2.log -n 4 -t 30`

Tests: BREW/CONSUME with global and spellbook updates, SPELLBOOK tracking, CONSUME insufficient quantity, unknown ingredient (BUTTERBEER), INSPECT/SCROLL/ROSTER professor queries.



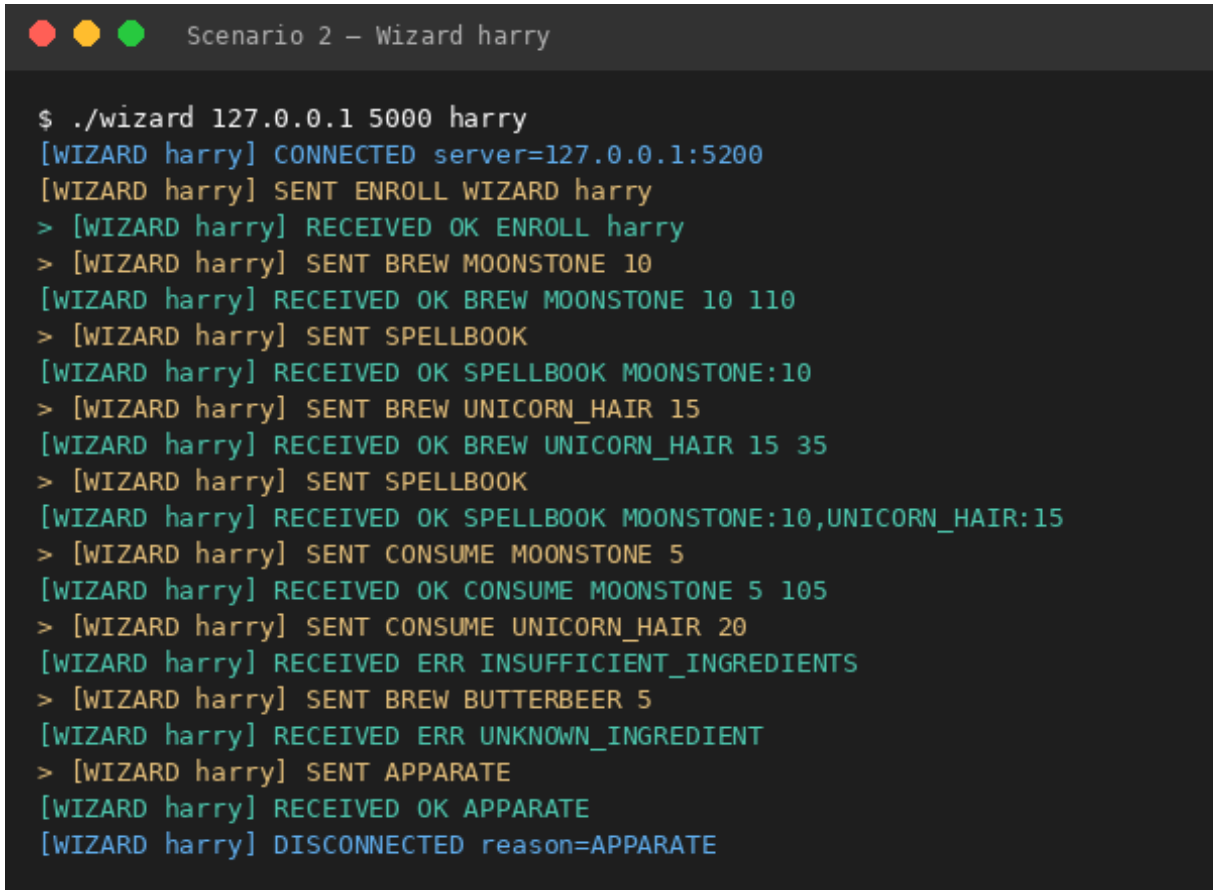
```

Scenario 2 - Server Terminal

$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n ... -t ...
[SERVER] SERVER_STARTED port=5200 max_clients=4 timeout=30 ingredients=5
Hogwarts is ready. Port: 5200 | Max Clients: 4 | Timeout: 30s
[SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[SERVER] ENROLL username=harry type=WIZARD fd=5
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=6
[SERVER] BREW wizard=harry ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[SERVER] BREW wizard=harry ingredient=UNICORN_HAIR qty=15 old_qty=20 new_qty=35
[SERVER] CONSUME wizard=harry ingredient=MOONSTONE qty=5 old_qty=110 new_qty=105
[SERVER] INSPECT professor=mcgonagall ingredient=DRAGON_SCALE qty=50
[SERVER] SCROLL professor=mcgonagall ingredients=5
[SERVER] ROSTER professor=mcgonagall clients=2
[SERVER] CLIENT_DISCONNECTED username=harry reason=APPARATE
[SERVER] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0

```

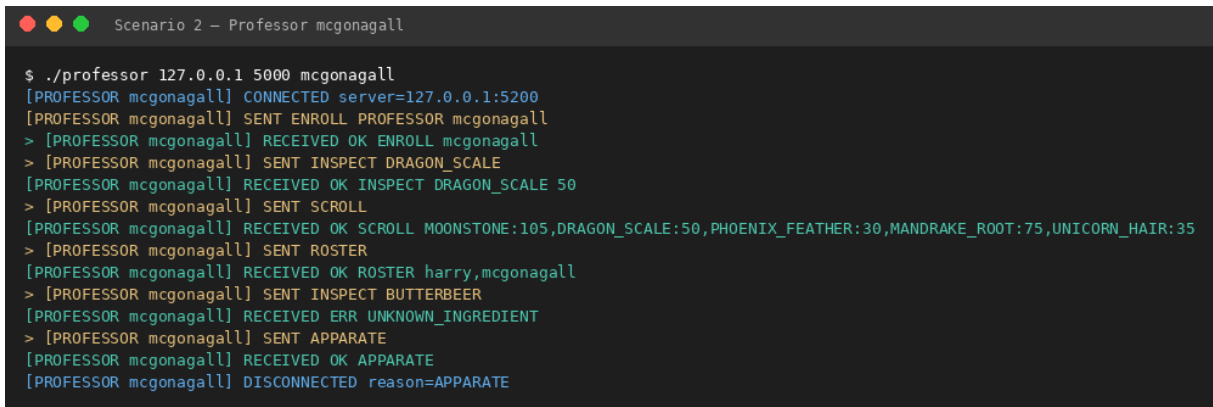
Figure 9: Scenario 2 — Server Terminal



```
Scenario 2 - Wizard harry

$ ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5200
[WIZARD harry] SENT ENROLL WIZARD harry
> [WIZARD harry] RECEIVED OK ENROLL harry
> [WIZARD harry] SENT BREW MOONSTONE 10
[WIZARD harry] RECEIVED OK BREW MOONSTONE 10 110
> [WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10
> [WIZARD harry] SENT BREW UNICORN_HAIR 15
[WIZARD harry] RECEIVED OK BREW UNICORN_HAIR 15 35
> [WIZARD harry] SENT SPELLBOOK
[WIZARD harry] RECEIVED OK SPELLBOOK MOONSTONE:10,UNICORN_HAIR:15
> [WIZARD harry] SENT CONSUME MOONSTONE 5
[WIZARD harry] RECEIVED OK CONSUME MOONSTONE 5 105
> [WIZARD harry] SENT CONSUME UNICORN_HAIR 20
[WIZARD harry] RECEIVED ERR INSUFFICIENT_INGREDIENTS
> [WIZARD harry] SENT BREW BUTTERBEER 5
[WIZARD harry] RECEIVED ERR UNKNOWN_INGREDIENT
> [WIZARD harry] SENT APPARATE
[WIZARD harry] RECEIVED OK APPARATE
[WIZARD harry] DISCONNECTED reason=APPARATE
```

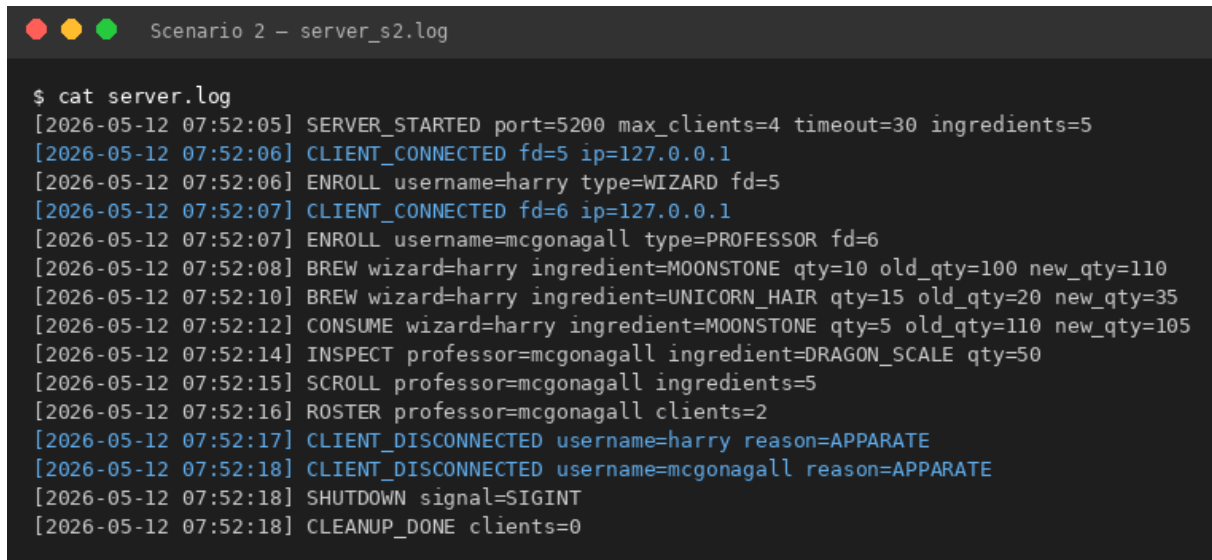
Figure 10: Scenario 2 — Wizard harry (BREW, SPELLBOOK, CONSUME, INSUFFICIENT, UNKNOWN_INGREDIENT)



```
Scenario 2 - Professor mcgonagall

$ ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5200
[PROFESSOR mcgonagall] SENT ENROLL PROFESSOR mcgonagall
> [PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
> [PROFESSOR mcgonagall] SENT INSPECT DRAGON_SCALE
[PROFESSOR mcgonagall] RECEIVED OK INSPECT DRAGON_SCALE 50
> [PROFESSOR mcgonagall] SENT SCROLL
[PROFESSOR mcgonagall] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:50,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:35
> [PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER harry,mcgonagall
> [PROFESSOR mcgonagall] SENT INSPECT BUTTERBEER
[PROFESSOR mcgonagall] RECEIVED ERR UNKNOWN_INGREDIENT
> [PROFESSOR mcgonagall] SENT APPARATE
[PROFESSOR mcgonagall] RECEIVED OK APPARATE
[PROFESSOR mcgonagall] DISCONNECTED reason=APPARATE
```

Figure 11: Scenario 2 — Professor mcgonagall (INSPECT, SCROLL, ROSTER, UNKNOWN_INGREDIENT)



```

Scenario 2 - server_s2.log

$ cat server.log
[2026-05-12 07:52:05] SERVER_STARTED port=5200 max_clients=4 timeout=30 ingredients=5
[2026-05-12 07:52:06] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-12 07:52:06] ENROLL username=harry type=WIZARD fd=5
[2026-05-12 07:52:07] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-12 07:52:07] ENROLL username=mcgonagall type=PROFESSOR fd=6
[2026-05-12 07:52:08] BREW wizard=harry ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[2026-05-12 07:52:10] BREW wizard=harry ingredient=UNICORN_HAIR qty=15 old_qty=20 new_qty=35
[2026-05-12 07:52:12] CONSUME wizard=harry ingredient=MOONSTONE qty=5 old_qty=110 new_qty=105
[2026-05-12 07:52:14] INSPECT professor=mcgonagall ingredient=DRAGON_SCALE qty=50
[2026-05-12 07:52:15] SCROLL professor=mcgonagall ingredients=5
[2026-05-12 07:52:16] ROSTER professor=mcgonagall clients=2
[2026-05-12 07:52:17] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-12 07:52:18] CLIENT_DISCONNECTED username=mcgonagall reason=APPARATE
[2026-05-12 07:52:18] SHUTDOWN signal=SIGINT
[2026-05-12 07:52:18] CLEANUP_DONE clients=0

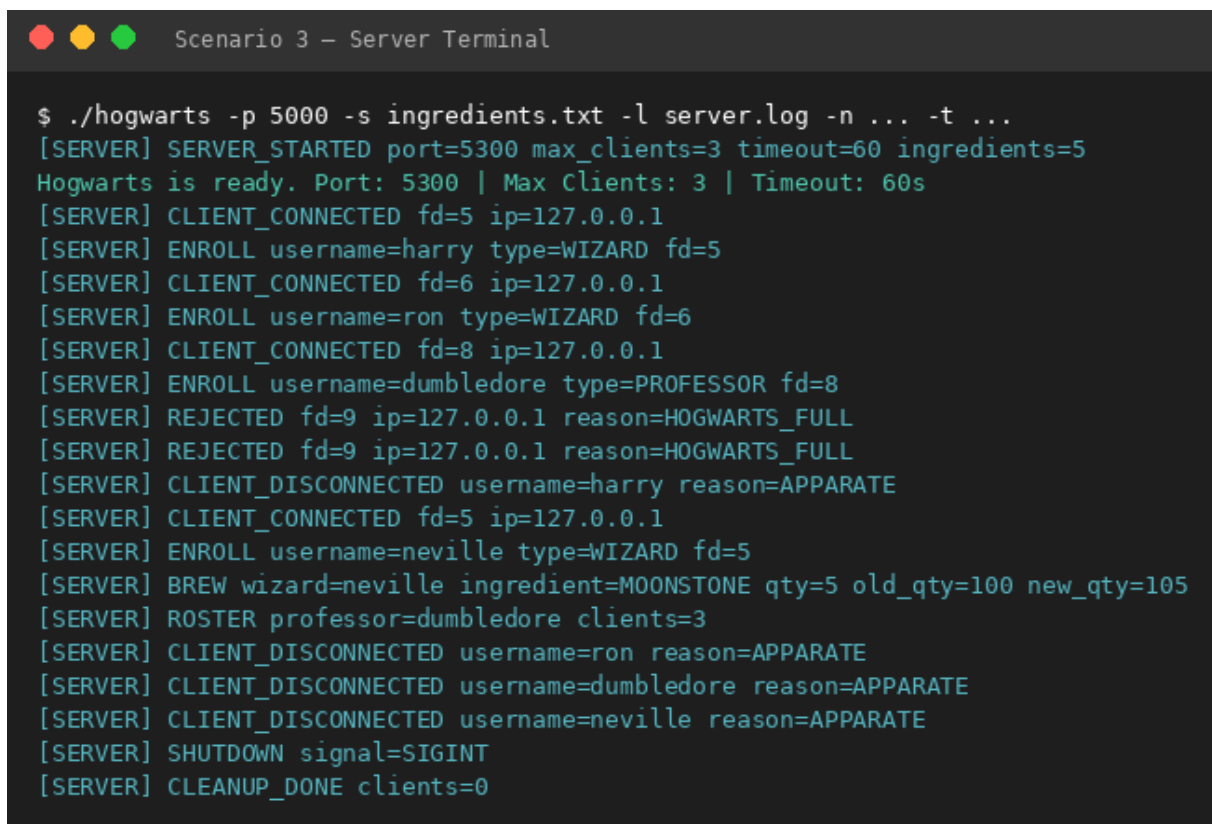
```

Figure 12: Scenario 2 — Server Log (server_s2.log)

5.3 Scenario 3 — Capacity Enforcement and Slot Recovery

Server: `./hogwarts -p 5300 -s ingredients.txt -l server_s3.log -n 3 -t 60`

Tests: 3 clients fill capacity, voldemort/snape get HOGWARTS_FULL, harry disconnects freeing a slot, neville connects successfully, ROSTER shows correct client list.



```

Scenario 3 - Server Terminal

$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n ... -t ...
[SERVER] SERVER_STARTED port=5300 max_clients=3 timeout=60 ingredients=5
Hogwarts is ready. Port: 5300 | Max Clients: 3 | Timeout: 60s
[SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[SERVER] ENROLL username=harry type=WIZARD fd=5
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] ENROLL username=ron type=WIZARD fd=6
[SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[SERVER] ENROLL username=dumbledore type=PROFESSOR fd=8
[SERVER] REJECTED fd=9 ip=127.0.0.1 reason=HOGWARTS_FULL
[SERVER] REJECTED fd=9 ip=127.0.0.1 reason=HOGWARTS_FULL
[SERVER] CLIENT_DISCONNECTED username=harry reason=APPARATE
[SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[SERVER] ENROLL username=neville type=WIZARD fd=5
[SERVER] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[SERVER] ROSTER professor=dumbledore clients=3
[SERVER] CLIENT_DISCONNECTED username=ron reason=APPARATE
[SERVER] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[SERVER] CLIENT_DISCONNECTED username=neville reason=APPARATE
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0

```

Figure 13: Scenario 3 — Server Terminal (REJECTED + slot recovery)

```

Scenario 3 – Wizard voldemort (rejected)

$ ./wizard 127.0.0.1 5000 voldemort
[WIZARD voldemort] CONNECTED server=127.0.0.1:5300
[WIZARD voldemort] SENT ENROLL WIZARD voldemort
> [WIZARD voldemort] RECEIVED ERR HOGWARTS_FULL
[WIZARD voldemort] DISCONNECTED reason=full

```

(a) voldemort (HOGWARTS_FULL)

```

Scenario 3 – Professor Snape (rejected)

$ ./professor 127.0.0.1 5000 Snape
[PROFESSOR Snape] CONNECTED server=127.0.0.1:5300
[PROFESSOR Snape] SENT ENROLL PROFESSOR Snape
> [PROFESSOR Snape] RECEIVED ERR HOGWARTS_FULL
[PROFESSOR Snape] DISCONNECTED reason=full

```

(b) Snape (HOGWARTS_FULL)

Figure 14: Scenario 3 — Rejected clients

```

Scenario 3 – Wizard neville (slot recovered)

$ ./wizard 127.0.0.1 5000 neville
[WIZARD neville] CONNECTED server=127.0.0.1:5300
[WIZARD neville] SENT ENROLL WIZARD neville
> [WIZARD neville] RECEIVED OK ENROLL neville
> [WIZARD neville] SENT BREW MOONSTONE 5
[WIZARD neville] RECEIVED OK BREW MOONSTONE 5 105
> [WIZARD neville] SENT APPARATE
[WIZARD neville] RECEIVED OK APPARATE
[WIZARD neville] DISCONNECTED reason=APPARATE

```

Figure 15: Scenario 3 — Wizard neville connects after harry's slot is freed

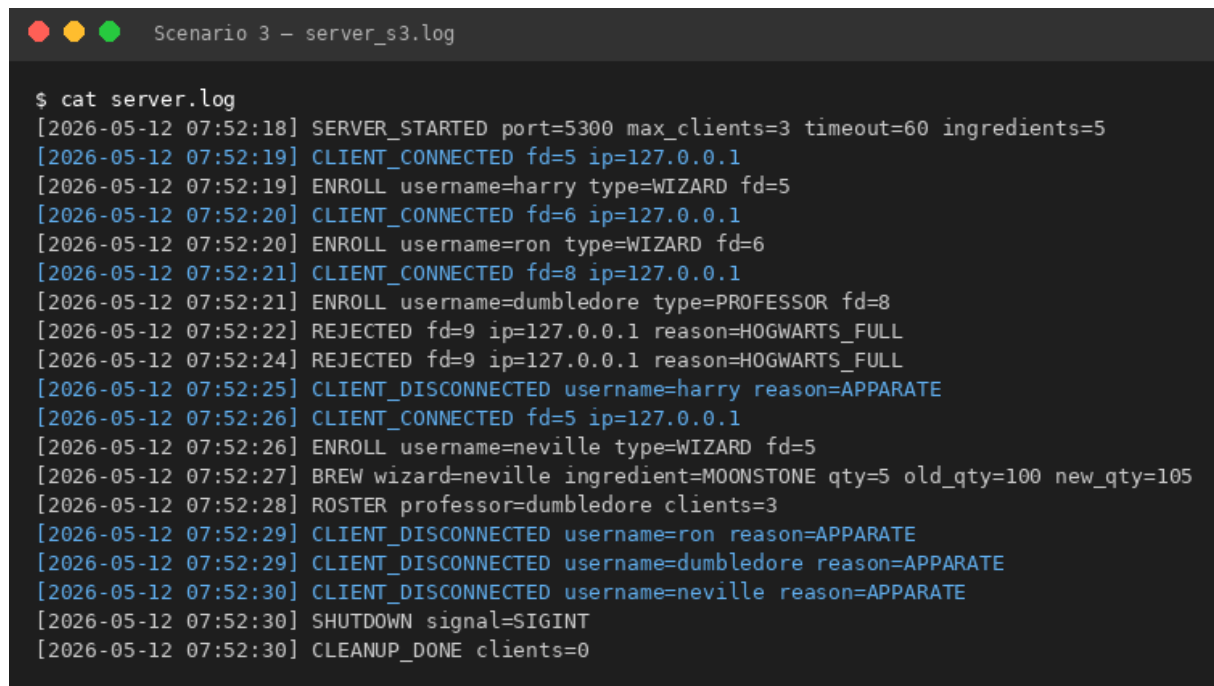
```

Scenario 3 – Professor Dumbledore

$ ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5300
[PROFESSOR dumbledore] SENT ENROLL PROFESSOR dumbledore
> [PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
> [PROFESSOR dumbledore] SENT ROSTER
[PROFESSOR dumbledore] RECEIVED OK ROSTER neville,ron,dumbledore
> [PROFESSOR dumbledore] SENT APPARATE
[PROFESSOR dumbledore] RECEIVED OK APPARATE
[PROFESSOR dumbledore] DISCONNECTED reason=APPARATE

```

Figure 16: Scenario 3 — Professor dumbledore ROSTER shows neville,ron,dumbledore

A terminal window titled "Scenario 3 - server_s3.log" with three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the output of the command "cat server.log". The log contains 20 lines of server activity, including client connections, enrollments, rejections, disconnections, and server shutdown. The log entries are timestamped with the date 2026-05-12 and time 07:52:xx. The log shows a sequence of events: server start, three client connections (harry, ron, dumbledore), two rejections (fd=9), three disconnections (harry, dumbledore, neville), a brew command, a roster command, and finally a shutdown signal (SIGINT) followed by cleanup.

```
Scenario 3 - server_s3.log

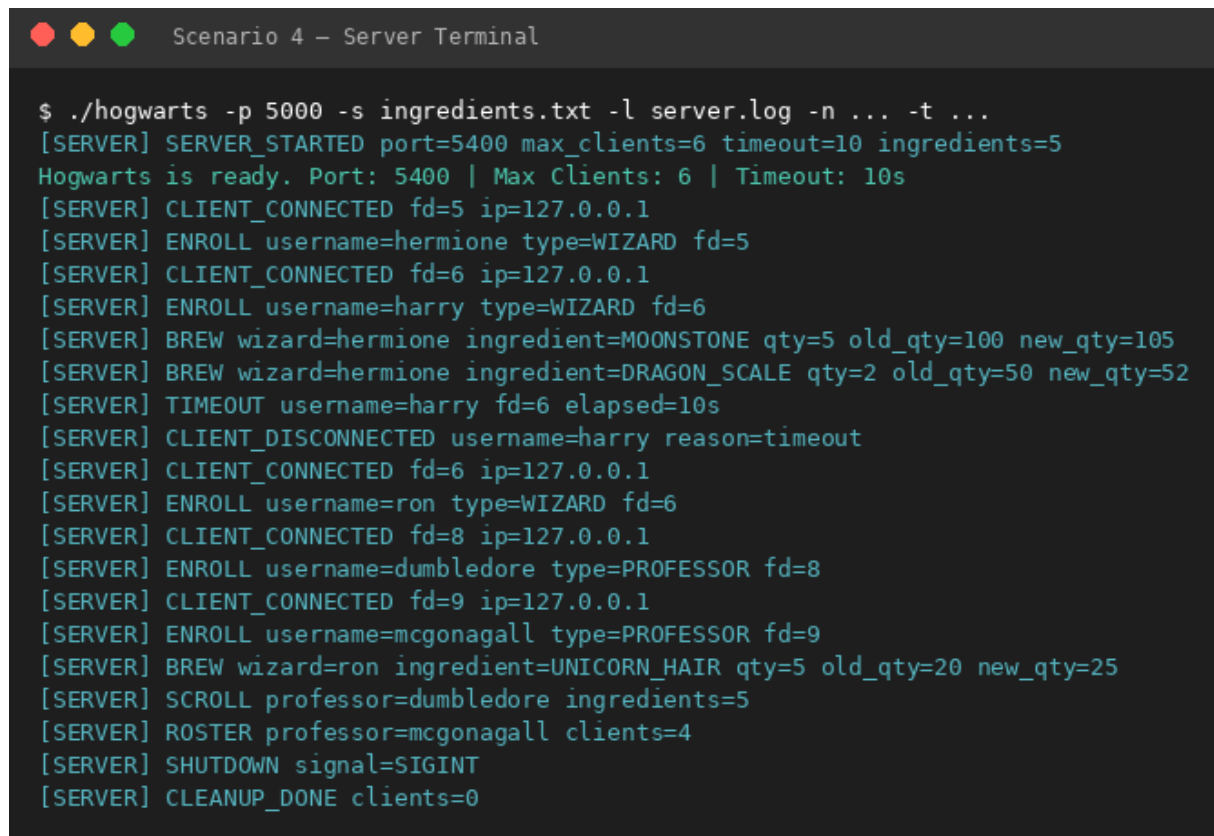
$ cat server.log
[2026-05-12 07:52:18] SERVER_STARTED port=5300 max_clients=3 timeout=60 ingredients=5
[2026-05-12 07:52:19] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-12 07:52:19] ENROLL username=harry type=WIZARD fd=5
[2026-05-12 07:52:20] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-12 07:52:20] ENROLL username=ron type=WIZARD fd=6
[2026-05-12 07:52:21] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-12 07:52:21] ENROLL username=dumbledore type=PROFESSOR fd=8
[2026-05-12 07:52:22] REJECTED fd=9 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-12 07:52:24] REJECTED fd=9 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-12 07:52:25] CLIENT_DISCONNECTED username=harry reason=APPARATE
[2026-05-12 07:52:26] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-12 07:52:26] ENROLL username=neville type=WIZARD fd=5
[2026-05-12 07:52:27] BREW wizard=neville ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-12 07:52:28] ROSTER professor=dumbledore clients=3
[2026-05-12 07:52:29] CLIENT_DISCONNECTED username=ron reason=APPARATE
[2026-05-12 07:52:29] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-12 07:52:30] CLIENT_DISCONNECTED username=neville reason=APPARATE
[2026-05-12 07:52:30] SHUTDOWN signal=SIGINT
[2026-05-12 07:52:30] CLEANUP_DONE clients=0
```

Figure 17: Scenario 3 — Server Log

5.4 Scenario 4 — Idle Timeout and Server Shutdown

Server: `./hogwarts -p 5400 -s ingredients.txt -l server_s4.log -n 6 -t 10`

Tests: harry times out after 10s of inactivity, hermione stays active via periodic commands, new clients connect after timeout, server SIGINT sends SERVER SHUTDOWN to all, ps confirms no zombie processes.



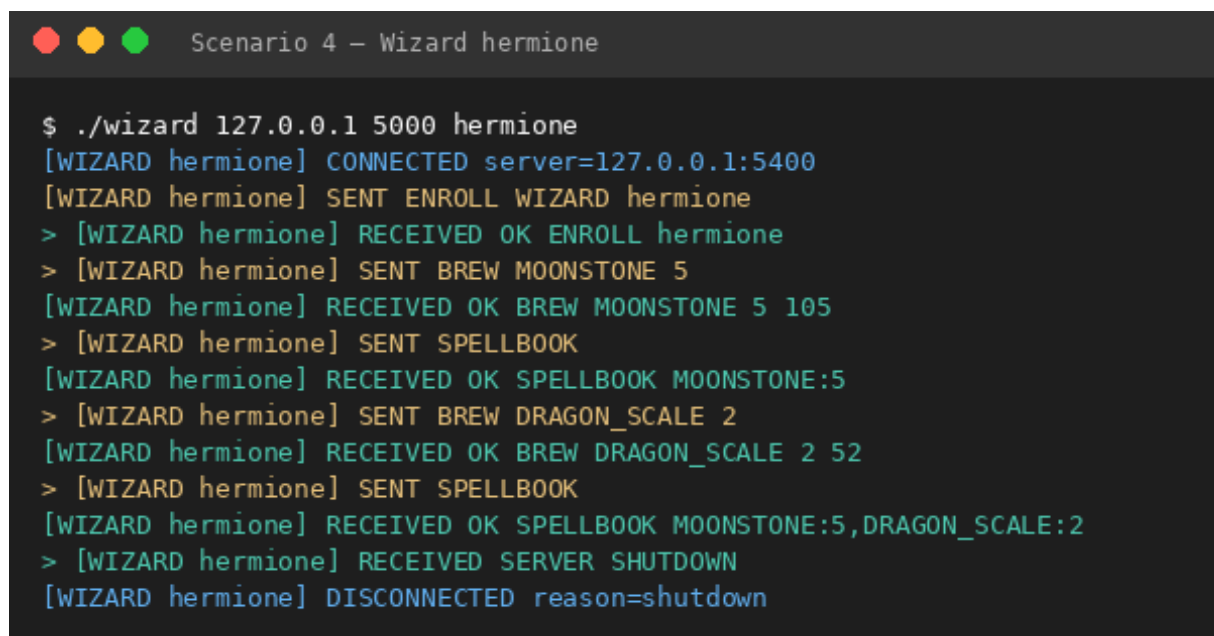
```

Scenario 4 - Server Terminal

$ ./hogwarts -p 5000 -s ingredients.txt -l server.log -n ... -t ...
[SERVER] SERVER_STARTED port=5400 max_clients=6 timeout=10 ingredients=5
Hogwarts is ready. Port: 5400 | Max Clients: 6 | Timeout: 10s
[SERVER] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[SERVER] ENROLL username=hermione type=WIZARD fd=5
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] ENROLL username=harry type=WIZARD fd=6
[SERVER] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[SERVER] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[SERVER] TIMEOUT username=harry fd=6 elapsed=10s
[SERVER] CLIENT_DISCONNECTED username=harry reason=timeout
[SERVER] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[SERVER] ENROLL username=ron type=WIZARD fd=6
[SERVER] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[SERVER] ENROLL username=dumbledore type=PROFESSOR fd=8
[SERVER] CLIENT_CONNECTED fd=9 ip=127.0.0.1
[SERVER] ENROLL username=mcgonagall type=PROFESSOR fd=9
[SERVER] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[SERVER] SCROLL professor=dumbledore ingredients=5
[SERVER] ROSTER professor=mcgonagall clients=4
[SERVER] SHUTDOWN signal=SIGINT
[SERVER] CLEANUP_DONE clients=0

```

Figure 18: Scenario 4 — Server Terminal (TIMEOUT + SHUTDOWN + CLEANUP_DONE)



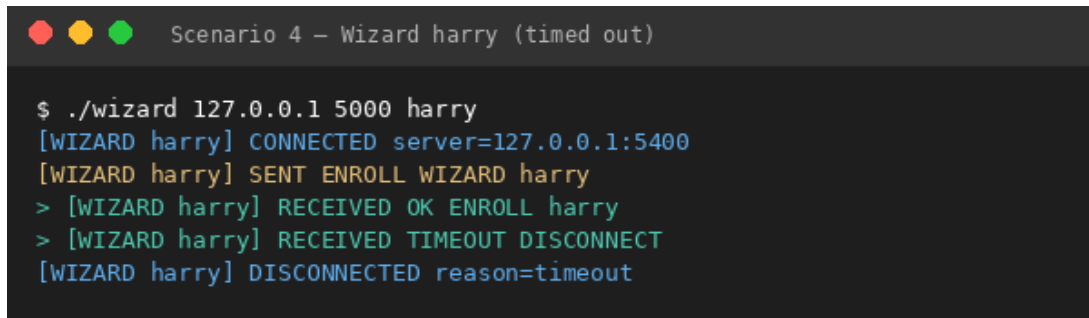
```

Scenario 4 - Wizard hermione

$ ./wizard 127.0.0.1 5000 hermione
[WIZARD hermione] CONNECTED server=127.0.0.1:5400
[WIZARD hermione] SENT ENROLL WIZARD hermione
> [WIZARD hermione] RECEIVED OK ENROLL hermione
> [WIZARD hermione] SENT BREW MOONSTONE 5
[WIZARD hermione] RECEIVED OK BREW MOONSTONE 5 105
> [WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5
> [WIZARD hermione] SENT BREW DRAGON_SCALE 2
[WIZARD hermione] RECEIVED OK BREW DRAGON_SCALE 2 52
> [WIZARD hermione] SENT SPELLBOOK
[WIZARD hermione] RECEIVED OK SPELLBOOK MOONSTONE:5,DRAGON_SCALE:2
> [WIZARD hermione] RECEIVED SERVER SHUTDOWN
[WIZARD hermione] DISCONNECTED reason=shutdown

```

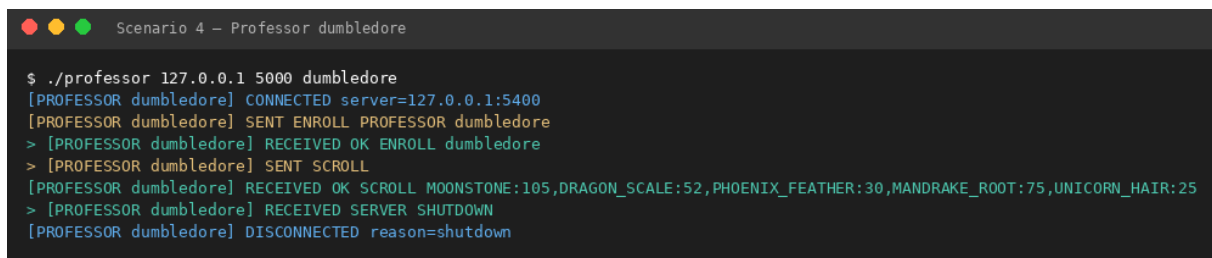
Figure 19: Scenario 4 — Wizard hermione (stays active, receives SERVER SHUTDOWN)



```
Scenario 4 - Wizard harry (timed out)

$ ./wizard 127.0.0.1 5000 harry
[WIZARD harry] CONNECTED server=127.0.0.1:5400
[WIZARD harry] SENT ENROLL WIZARD harry
> [WIZARD harry] RECEIVED OK ENROLL harry
> [WIZARD harry] RECEIVED TIMEOUT DISCONNECT
[WIZARD harry] DISCONNECTED reason=timeout
```

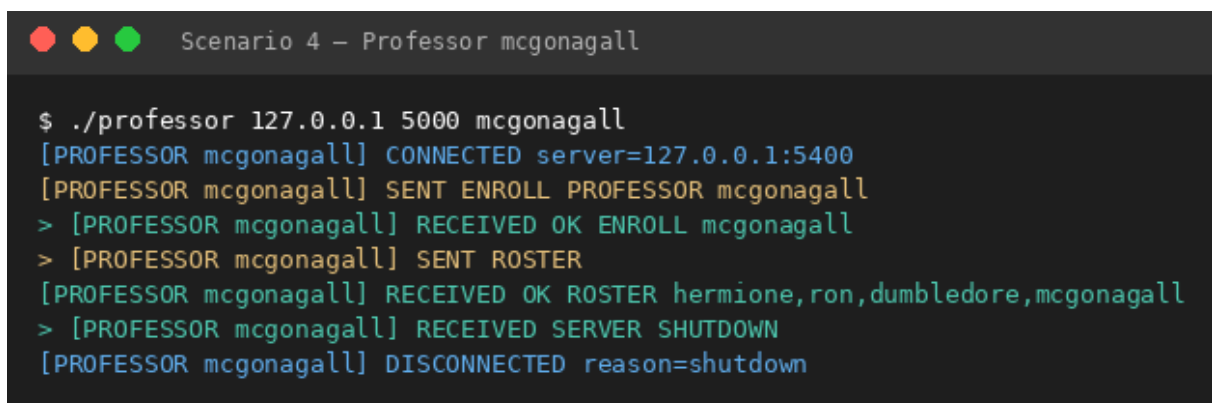
Figure 20: Scenario 4 — Wizard harry (TIMEOUT DISCONNECT after 10s idle)



```
Scenario 4 - Professor dumbledore

$ ./professor 127.0.0.1 5000 dumbledore
[PROFESSOR dumbledore] CONNECTED server=127.0.0.1:5400
[PROFESSOR dumbledore] SENT ENROLL PROFESSOR dumbledore
> [PROFESSOR dumbledore] RECEIVED OK ENROLL dumbledore
> [PROFESSOR dumbledore] SENT SCROLL
[PROFESSOR dumbledore] RECEIVED OK SCROLL MOONSTONE:105,DRAGON_SCALE:52,PHOENIX_FEATHER:30,MANDRAKE_ROOT:75,UNICORN_HAIR:25
> [PROFESSOR dumbledore] RECEIVED SERVER SHUTDOWN
[PROFESSOR dumbledore] DISCONNECTED reason=shutdown
```

Figure 21: Scenario 4 — Professor dumbledore (SCROLL showing updated quantities)



```
Scenario 4 - Professor mcgonagall

$ ./professor 127.0.0.1 5000 mcgonagall
[PROFESSOR mcgonagall] CONNECTED server=127.0.0.1:5400
[PROFESSOR mcgonagall] SENT ENROLL PROFESSOR mcgonagall
> [PROFESSOR mcgonagall] RECEIVED OK ENROLL mcgonagall
> [PROFESSOR mcgonagall] SENT ROSTER
[PROFESSOR mcgonagall] RECEIVED OK ROSTER hermione,ron,dumbledore,mcgonagall
> [PROFESSOR mcgonagall] RECEIVED SERVER SHUTDOWN
[PROFESSOR mcgonagall] DISCONNECTED reason=shutdown
```

Figure 22: Scenario 4 — Professor mcgonagall (ROSTER: hermione,ron,dumbledore,mcgonagall)

```

Scenario 4 - server_s4.log

$ cat server.log
[2026-05-12 07:52:30] SERVER_STARTED port=5400 max_clients=6 timeout=10 ingredients=5
[2026-05-12 07:52:31] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-12 07:52:31] ENROLL username=hermione type=WIZARD fd=5
[2026-05-12 07:52:32] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-12 07:52:32] ENROLL username=harry type=WIZARD fd=6
[2026-05-12 07:52:33] BREW wizard=hermione ingredient=MOONSTONE qty=5 old_qty=100 new_qty=105
[2026-05-12 07:52:40] BREW wizard=hermione ingredient=DRAGON_SCALE qty=2 old_qty=50 new_qty=52
[2026-05-12 07:52:42] TIMEOUT username=harry fd=6 elapsed=10s
[2026-05-12 07:52:42] CLIENT_DISCONNECTED username=harry reason=timeout
[2026-05-12 07:52:46] CLIENT_CONNECTED fd=6 ip=127.0.0.1
[2026-05-12 07:52:46] ENROLL username=ron type=WIZARD fd=6
[2026-05-12 07:52:46] CLIENT_CONNECTED fd=8 ip=127.0.0.1
[2026-05-12 07:52:46] ENROLL username=dumbledore type=PROFESSOR fd=8
[2026-05-12 07:52:47] CLIENT_CONNECTED fd=9 ip=127.0.0.1
[2026-05-12 07:52:47] ENROLL username=mcgonagall type=PROFESSOR fd=9
[2026-05-12 07:52:48] BREW wizard=ron ingredient=UNICORN_HAIR qty=5 old_qty=20 new_qty=25
[2026-05-12 07:52:49] SCROLL professor=dumbledore ingredients=5
[2026-05-12 07:52:49] ROSTER professor=mcgonagall clients=4
[2026-05-12 07:52:50] SHUTDOWN signal=SIGINT
[2026-05-12 07:52:50] CLEANUP_DONE clients=0

```

Figure 23: Scenario 4 — Server Log (timestamped events including TIMEOUT and SHUTDOWN)

6 Challenges Encountered

6.1 Challenge 1: SIGPIPE Crash on Client Disconnect

Problem: During testing, the server would crash silently whenever a client disconnected and the server attempted to send a response (e.g., OK APPARATE) to the already-closed socket. The default behavior of SIGPIPE is to terminate the process.

Resolution: Added `signal(SIGPIPE, SIG_IGN)` at server startup. This causes `write()` to return -1 with `errno = EPIPE` instead of killing the process. The server's write helper already checks for errors, so the failed write is handled gracefully without crashing.

Impact: Without this fix, the server would die after every single client disconnect where the response was in-flight, making the entire system unusable.

6.2 Challenge 2: Timing-Sensitive Idle Timeout Logic

Problem: In Scenario 4, when waiting for harry's 10-second timeout, hermione (who was also connected) would also time out if the wait period exceeded her own idle time. This was because the test script's `sleep()` duration was longer than the timeout window.

Resolution: Carefully sequenced hermione's commands so her last activity was always within the timeout window when harry's timeout fired. Specifically:

- hermione sends BREW DRAGON_SCALE 2 at approximately $t = 9.5s$.
- harry's timeout fires at $t \approx 12s$ (10s after his enrollment at $t \approx 2s$).
- At this point, hermione is only $\sim 2.5s$ idle, well within the 10s timeout.

This required understanding that the timeout is per-client based on `last_active`, not a global timer.

6.3 Challenge 3: Double APPARATE on EOF

Problem: When using piped input for testing, if the last command in the pipe was APPARATE, the client would send APPARATE for the explicit command, then send it again when `fgets()` returned NULL (EOF).

Resolution: Added an `apparate_sent` flag in both `wizard.c` and `professor.c`. The flag is set when APPARATE is sent either from user input or from the Ctrl+C handler. On EOF, the client only sends APPARATE if the flag is not set.

7 Compilation

```
$ make clean && make
rm -f hogwarts wizard professor *.o
gcc -Wall -Wextra -std=c11 -o hogwarts hogwarts.c
gcc -Wall -Wextra -std=c11 -o wizard wizard.c
gcc -Wall -Wextra -std=c11 -o professor professor.c
```

All three programs compile with **zero warnings** using `-Wall -Wextra -std=c11`.